INFORME SOBE LAS FUNCIONES DE ACTIVACIÓN

1. Introducción y Definición

En el ámbito del aprendizaje profundo (*Deep Learning*), las **funciones de activación** son transformaciones matemáticas que se aplican sobre la suma ponderada de las entradas de una neurona artificial.

Su función primaria es **introducir no linealidad** en el modelo. Dado que la suma de operaciones lineales sigue siendo una combinación lineal, un modelo profundo compuesto únicamente por transformaciones afines resultaría matemáticamente equivalente a una regresión lineal simple (y = Wx + b), reduciendo drásticamente su capacidad de representación. La presencia de no linealidades habilita a la red neuronal para aproximar funciones complejas, conforme al *Teorema de Aproximación Universal*.

2. Funciones de Activación

1. Lineal / Identidad

- **Fórmula:** $f(z) = z$
- **Rango:** $(-\infty, \infty)$
- **Uso principal:** Capas de salida en problemas de **regresión continua**. No modifica la suma ponderada.

2. Escalón de Heaviside (Binary Step)

- **Fórmula:** $f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$
- **Rango:** $\{0, 1\}$
- **Uso principal:** Perceptrón simple (histórico). Inviable para redes profundas debido a que su derivada es 0 en casi todo su dominio.

3. Sigmoide (Logística)

- **Fórmula:** $f(z) = \frac{1}{1+e^{-z}}$
- **Rango:** $(0, 1)$
- **Uso principal:** Capa de salida en **clasificación binaria**. Genera salidas interpretables como probabilidades.

4. Tangente Hiperbólica (TanH)

- **Fórmula:** $f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- **Rango:** $(-1, 1)$
- **Uso principal:** Capas ocultas en Redes Recurrentes (RNN, LSTM). Está centrada en cero.

5. Softmax

- **Fórmula:** $f(z_i) = \frac{e^{z_i}}{\sum_{k-1}^{K} e^{z_k}}$
- **Rango:** $(0, 1)$, con $\sum f(z_i) = 1$
- **Uso principal:** Capa de salida en **clasificación multiclase**. Convierte un vector de números en una distribución de probabilidad.

6. Softsign

- **Fórmula:** $f(z) = \frac{z}{1 + |z|}$
- **Rango:** $(-1, 1)$
- **Uso principal:** Alternativa a TanH. Se satura más lentamente que TanH, aunque es menos utilizada en arquitecturas modernas.

7. ReLU (Rectified Linear Unit)

- **Fórmula:** $f(z) = \max(0, z)$
- **Rango:** $[0, \infty)$
- **Uso principal:** Estándar en redes convolucionales (CNN) y perceptrones multicapa (MLP) por su alta eficiencia computacional.

8. Leaky ReLU

- **Fórmula:** $f(z) = \max(\alpha z, z) \quad (\alpha \approx 0.01)$
- **Rango:** $(-\infty, \infty)$
- **Uso principal:** Evita el problema de neuronas muertas (*Dying ReLU*) permitiendo una pequeña pendiente constante cuando $z < 0$.

9. PReLU (Parametric ReLU)

- **Fórmula:** $f(z) = \max(\alpha z, z)$
- **Rango:** $(-\infty, \infty)$
- **Uso principal:** Igual a Leaky ReLU, pero la pendiente α es un **parámetro aprendible** mediante retropropagación durante el entrenamiento.

10. RReLU (Randomized Leaky ReLU)

- **Fórmula:** $f(z) = \max(\alpha_{i,j} z, z) \quad \text{donde } \alpha_{i,j} \sim U(l, u)$
- **Rango:** $(-\infty, \infty)$
- **Uso principal:** Variación donde α se muestrea al azar de una distribución uniforme durante el entrenamiento y se fija en la inferencia. Funciona como regularizador.

11. ReLU6

- **Fórmula:** $f(z) = \min(\max(0, z), 6)$
- **Rango:** $[0, 6]$
- **Uso principal:** Diseñada para arquitecturas móviles y embebidas (**MobileNet**). Previene la pérdida de precisión al trabajar con aritmética de punto flotante de baja precisión (FP16/INT8).

12. SReLU (S-shaped ReLU)

- **Fórmula:** Función por partes definida por dos umbrales (t_l, t_r) y dos pendientes (a_l, a_r) aprendibles.
- **Rango:** $(-\infty, \infty)$
- **Uso principal:** Permite aprender transformaciones no lineales convexas y no convexas complejas.

13. ELU (Exponential Linear Unit)

- **Fórmula:** $f(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$
- **Rango:** $(-\alpha, \infty)$
- **Uso principal:** Suaviza la zona negativa, reduciendo el sesgo medio de las activaciones hacia cero y acelerando la convergencia frente a ReLU.

14. SELU (Scaled Exponential Linear Unit)

- **Fórmula:** $f(z) = \lambda \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$
  (con $\lambda \approx 1.0507$ y $\alpha \approx 1.6732$)
- **Rango:** $(-\lambda\alpha, \infty)$
- **Uso principal:** Base de las **Self-Normalizing Neural Networks (SNNs)**. Permite que las activaciones se autonormalicen sin necesidad de *Batch Normalization*.

15. CELU (Continuously Differentiable ELU)

- **Fórmula:** $f(z) = \max(0, z) + \min\left(0, \alpha\left(e^{z/\alpha} - 1\right)\right)$
- **Rango:** $(-\alpha, \infty)$
- **Uso principal:** Variante de ELU continuamente derivable en $z = 0$, útil cuando las derivadas de orden superior son requeridas.

16. Softplus

- **Fórmula:** $f(z) = \ln(1 + e^z)$
- **Rango:** $(0, \infty)$
- **Uso principal:** Versión suave e infinitamente derivable de ReLU. Usada en modelos probabilísticos y redes Bayesianas.

17. GELU (Gaussian Error Linear Unit)

- **Fórmula:** $f(z) = z \cdot \Phi(z) \approx 0.5z \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right)$
- **Rango:** $(\approx -0.17, \infty)$
- **Uso principal:** Estándar de la industria en **Transformers** (GPT-3/4, BERT, ViT). Combina la rectificación con el comportamiento de la distribución normal.

18. Swish / SiLU (Sigmoid Linear Unit)

- **Fórmula:** $f(z) = z \cdot \sigma(\beta z) = \frac{z}{1+e^{-\beta z}}$
- **Rango:** $(\approx -0.28, \infty)$
- **Uso principal:** Utilizada en arquitecturas de visión como **EfficientNet** y modelos **YOLO**. Descubierta mediante AutoML por Google.

19. Hard-Swish (H-Swish)

- **Fórmula:** $f(z) = z \cdot \frac{\text{ReLU6}(z+3)}{6}$
- **Rango:** $(\approx -0.375, \infty)$
- **Uso principal:** Variante optimizada computacionalmente de Swish para dispositivos móviles (**MobileNetV3**), reemplazando el cálculo del exponente por operaciones lineales.

20. Mish

- **Fórmula:** $f(z) = z \cdot \tanh(\text{softplus}(z)) = z \cdot \tanh(\ln(1 + e^z))$
- **Rango:** $(\approx -0.31, \infty)$
- **Uso principal:** Muy popular en modelos de detección de objetos como **YOLOv4**. Presenta una curva suave no monotónica que mejora el flujo del gradiente.

21. Snake

- **Fórmula:** $f(z) = z + \frac{1}{a}\sin^2(az)$
- **Rango:** $(-\infty, \infty)$
- **Uso principal:** Diseñada para aprender **patrones periódicos y cíclicos** (series temporales, audio y física informada por redes neuronales - PINNs).

22. Maxout

- **Fórmula:** $f(x) = \max_k(w_k^T x + b_k)$
- **Rango:** Depends on inputs
- **Uso principal:** Pondera un conjunto de k combinaciones afines para aproximar cualquier función convexa. Es altamente flexible pero duplica/multiplica el número de parámetros de la capa.